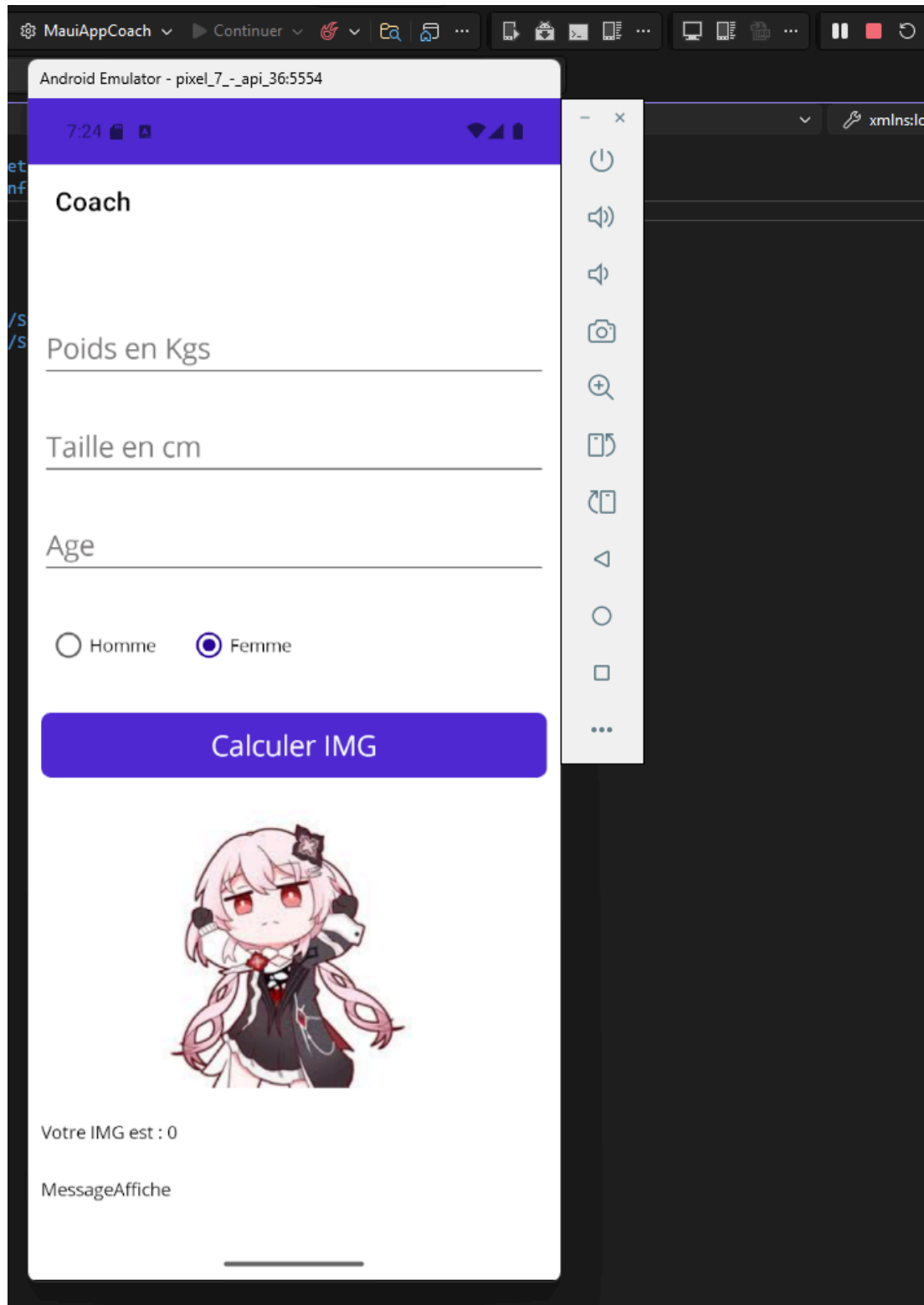


En deuxième année de BTS nous avons travaillé sur une application mobile de calcul d'IMG appelée MauiAppCoach.

Comme le nom l'indique l'application a été fait sous .NET MAUI et codé en c# .

.NET MAUI permet la création d'applications android ios ou encore windows.

Une émulation de l'outil sur lequel nous allons créer l'application peut même être simulée pour qu'on puisse visualiser à l'avance ce que nous sommes en train de créer .



L'affichage de l'Application est codé sur un fichier Xaml "MainPage.xaml"

Ici on y retrouve tout le visuel du site : Entry (champ de texte) , RadioButton (les boutons à cocher) , Button (le bouton) , Image (l'image) et les simples labels qui affiche le texte demandé. Il y a aussi dans ce fichier la disposition de visuel (verticalité et horizontalité) avec VerticalStackLayout qui entoure toute la page pour que le site s'affiche à la verticale et HorizontalStackLayout qui entoure RadioButton pour que les coche soi côte à côte.

```
<VerticalStackLayout VerticalOptions="Center" Spacing="25" Margin="10">
    <Entry
        TextColor="DarkBlue"
        FontSize="22"
        x:Name="entPoids"
        Placeholder="Poids en Kgs"
        PlaceholderColor="#7d7c7c"
        ClearButtonVisibility="WhileEditing"
        Keyboard="Numeric"
        VerticalOptions="Center">
    </Entry>
    <Entry
        TextColor="DarkBlue"
        FontSize="22"
        x:Name="entTaille"
        Placeholder="Taille en cm"
        PlaceholderColor="#7d7c7c"
        ClearButtonVisibility="WhileEditing"
        Keyboard="Numeric"
        VerticalOptions="Center">
    </Entry>
    <Entry
        TextColor="DarkBlue"
        FontSize="22"
        x:Name="entAge"
        Placeholder="Age"
        PlaceholderColor="#7d7c7c"
        ClearButtonVisibility="WhileEditing"
        Keyboard="Numeric"
        VerticalOptions="Center">
    </Entry>
    <HorizontalStackLayout VerticalOptions="Center" Spacing="25" Margin="5">
        <RadioButton Content="Homme"
            x:Name="rbHomme"
            GroupName="entSex"/>
        <RadioButton Content="Femme"
            x:Name="rbFemme"
            IsChecked="True"
            GroupName="entSex"/>
    </HorizontalStackLayout>
    <Button
        x:Name="btCalculer"
        FontSize="22"
        Text="Calculer IMG"
        SemanticProperties.Hint="Calculate IMG"
        HorizontalOptions="Fill"
        Clicked="btCalculer_Clicked"/>
    <Image
        x:Name="IMGimage"
        Source="evernight.png"/>
    <Label
        x:Name="IMGaffiche"
        Text="Votre IMG est : 0"/>
</VerticalStackLayout>
```

Ici nous pouvons aussi voir que tous ont un X : name pour les identifier plus tard

lorsqu'on devra faire réagir les différents bloc grâce au [mainPage.xaml.cs](#).

Ensuite il y a le fichier [Profil.cs](#) qui Contient la classe public avec toutes les variables utilisé et dans cette classe se trouve les méthode privée avec les fonctions pour calculez l'IMG avec les variables de la classe , l'autre Méthode à pour but d'afficher un message différent si l'IMG est trop bas, trop élevée ou parfait.

Il y a ensuite le constructeur qui donne les valeurs à toutes les variables puis les Geter qui retourne l'img et les messages.

```
1 référence
private string ResultatIMG()
{
    if (sexe == 1)
    {
        if (img < 15)
        {
            message = "Trop maigre !";
        }
        else if (img >= 15 && img <= 20)
        {
            message = "Parfait !!!!";
        }
        else
        {
            message = "Surpoids !";
        }
    }
    else if (sexe == 0)
    {
        if (img < 25)
        {
            message = "Trop maigre !";
        }
        else if (img >= 25 && img <= 30)
        {
            message = "Parfait !!!!";
        }
        else
        {
            message = "Surpoids !";
        }
    }
    else
    {
        throw new Exception("Problème détecté");
    }
    return message;
}
2 références 1/1 passing
public Profil(int sexe, int poids, int taille, int age)
{
    this.sexe = sexe;
    this.poids = poids;
    this.taille = taille;
    this.age = age;
    this.img = CalculIMG();
    this.message = ResultatIMG();
}

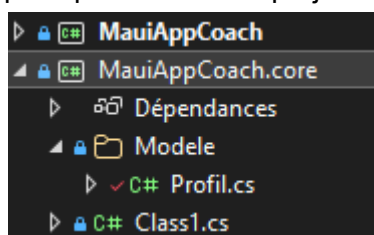
2 références 1/1 passing
public double GetImg()
{
    return img;
}
3 références
public string GetMessage()
{
    return message;
}
```

De plus nous avons aussi un fichier [MainPage.xaml.cs](#) précédemment cité ce fichier a pour utilité ,grâce au méthode créez ,d'appliquer les méthode du [profil.cs](#) sur les blocs du MainPage.xaml . Pour cela il faut premièrement appeler avec un using le dossier du fichier [Profil.cs](#) . Ensuite les méthodes se servent des x : name pour interagir avec les mainpage.xaml comme par exemple en utilisant le x : name du dernier label pour y afficher autre choses à l'appui d'un bouton ou encore pour effectuer la méthode du calcul de l'IMG à partir des données entrées dans le entry.

```
0 références
private void btCalculer_Clicked(object sender, EventArgs e)
{
    int sexe = 0;
    if (rbHomme.IsChecked)
    {
        sexe = 1;
    }
    else if (rbFemme.IsChecked)
    {
        sexe = 0;
    }
    Profil profil = new Profil(sexe, Int32.Parse(entPoids.Text), Int32.Parse(entTaille.Text), Int32.Parse(entAge.Text));
    double img = 0;
    img = profil.GetImg();
    try
    {
        IMGaffiche.Text = $"Votre IMG est de {img} !";
        MessageAffiche.Text = profil.GetMessage();
        if (profil.GetMessage() == "Trop maigre !")
        {
            IMGimage.Source = "furinal.png";
        }
        else if (profil.GetMessage() == "Parfait !!!!")
        {
            IMGimage.Source = "furina2.png";
        }
        else
        {
            IMGimage.Source = "furina3.png";
        }
    }
    catch (Exception ex)
    {
        DisplayAlertAsync("Erreur", "saisie incorrecte", "Ok");
    }
}
```

Enfin pour tester le calcul de ma méthode j'ai dû avoir recours au test unitaire. Cependant , Je me suis heurté à un problème difficile à résoudre, en effet mon projet .NET MAUI est en net10.0-android hors la quasi totalité des test unitaires sont en net10.0 . J'ai tenté de modifier directement depuis le csproj mais ça ne fonctionnait pas le test refusait de s'effectuer j'ai donc dû faire fonctionner le test d'une autre manière.

J'ai créé une bibliothèque dans la solution et j'y ai déplacé mon dossier modele contenant mon fichier [profil.cs](#) la bibliothèque étant en net 10.0 le test unitaire n'avait plus de problème.[me.ii](#) ne faut pas oublier de mettre en référence la bibliothèque avec le projet principale et avec le projet de test unitaire .



Ici la bibliothèque ce nome Maui App Coach.core

Le test unitaire maintenant fonctionnelle sert à tester si le calcul fonctionne comme prévu en donnant la réponse attendu et en donnant les valeurs nécessaire au calcul le test est effectué puis si le résultat attendu est atteint le test est positif sinon une erreur de calcul se trouve dans votre fonction .

